HPP Assignment 4

The Gravitational N-Body Problem
Parallel Implementation

Huang Yu Ting
Ho Yu LAU
Oscar Jansson
Clara Lindberg

High Performance Programming
Uppsala University
Spring 2026

Contents

1 The Problem **3**
1.1 Initial Problem . . . 3
1.2 Parallized Problem . . . 3

2 The Serial Solution **4**
2.1 Version 1 - Non-optimized . . . 4
2.2 Version 2 - Symmetric Force Optimization, Loop Unrolling, Constant Caching . . . 4
2.3 Version 3 - Vectorization . . . 5

3 The Parallel Solution **7**
3.1 Pthread: Workload imbalance . . . 7
3.2 The second approach . . . 8
3.3 Pthread: Race condition . . . 8
3.3.1 The second approach . . . 9
3.4 OpenMP solution . . . 10

4 Serial Performance and Discussion **12**
4.1 Method . . . 12
4.2 Performance results . . . 12
4.3 Performance Analysis . . . 13

5 Parallel Performance and Comparison **14**
5.1 Performance Comparison: Serial vs Parallel Versions . . . 15

6 References **17**

1 The Problem

1.1 Initial Problem

The objective of this assignment is to implement and optimize a serial simulation of the gravitational N-body problem in two spatial dimensions. The N-body problem describes the motion of N particles that interact through Newton's law of gravitation. Each particle is influenced by the gravitational force exerted by every other particle in the system.

The force acting on particle i is computed as the sum of the pairwise interactions with all other particles. To avoid numerical instability when particles are very close to each other, a softened force model using Plummer softening is applied. The equations of motion are integrated in time using the symplectic Euler method.

Since the force on each particle requires summation over all other particles, the computational complexity of the straightforward algorithm is $\mathcal{O}(N^2)$ per timestep. The goal of this assignment is therefore twofold: first, to implement a correct simulation that produces results consistent with the provided reference data; and second, to optimize the serial implementation to achieve the best possible performance while maintaining correctness and portability.

The program reads initial particle data from a binary file, simulates the system for a specified number of timesteps, and writes the final state to an output file in the same format. Performance measurements are used to analyze how execution time scales with N and to confirm the expected quadratic complexity.

1.2 Parallized Problem

The parallelized problem extends the serial N-body simulation by exploiting the fact that the computation of gravitational forces between particles is highly parallelizable. Since the dominant computational cost arises from the pairwise force calculation, which has a complexity of $\mathcal{O}(N^2)$ per timestep, the main objective of the parallelization is to distribute this workload across multiple threads in order to reduce execution time.

The force acting on each particle can be computed independently once the positions and masses of all particles are known for the current timestep. However, care must be taken when updating force accumulators to avoid race conditions. In particular, when symmetric force updates are used ($F_{ij} = -F_{ji}$), two particles are updated simultaneously, which introduces potential write conflicts in a multi-threaded environment.

To address this, the computational domain is divided among threads such that each thread is responsible for a subset of particles. The parallelization focuses on the outer loop over particles, while ensuring thread-safe updates of force arrays. Two parallel implementations were developed:

- A Pthreads-based implementation, where threads are created explicitly and assigned disjoint portions of the particle set. Synchronization is handled using appropriate thread coordination mechanisms.
- An OpenMP-based implementation, where compiler directives are used to parallelize the force calculation loop. Work-sharing constructs and reduction techniques are employed to prevent race conditions and ensure correctness.

The number of threads is specified as an input parameter, allowing the program to run with different levels of parallelism. This makes it possible to study the scalability of the implementation and to measure speedup relative to the optimized serial version.

The correctness of the parallel implementation is verified by comparing the final particle states with the serial version, ensuring that parallel execution does not alter the numerical results. Performance

is evaluated by measuring runtime for varying numbers of threads and problem sizes, and comparing the observed speedup to the ideal linear speedup.

2 The Serial Solution

This section outlines the implementation, the chosen data structures, and the algorithmic design decisions. The solution was developed by breaking the overall assignment into smaller, manageable subproblems, and progressively building on each partial solution. Below, three versions is explained corresponding to these different stages of implementation.

2.1 Version 1 - Non-optimized

The first version of the program serves as the baseline implementation. It computes all pairwise particle interactions using a straightforward double nested loop, without applying advanced optimization techniques. Only minimal modifications were introduced, such as replacing division operations with multiplication where possible and ensuring that the particle states remain consistent with the reference data. In this version, the nested loop contains an if condition to avoid self-interaction, which may introduce branch overhead and reduce execution efficiency. Moreover, each pairwise force is computed twice. Therefore, this baseline implementation provides a reference point for evaluating the effectiveness of subsequent optimizations.

2.2 Version 2 - Symmetric Force Optimization, Loop Unrolling, Constant Caching

In this version, the code was optimized using constant variable caching to reduce memory access, symmetric force optimization to eliminate redundant force computations, and loop unrolling to attempt reducing loop overhead.

By declaring variables that do not change as constants, constant variable caching allows frequently used value to be stored in cache rather than repeatedly accessing main memory. This reduces memory latency, speeds up execution, and improves overall performance, especially in loops or repeated computations. The runtime decreased from 11,474 ms to 11,246 ms for the case of N=3000 with a step size of 100.

The runtime of the simulation is directly proportional to the number of iterations in the nested for loops. To eliminate redundant computations of the same particle pair, the inner loop is started from `j = i + 1` instead of `j = 0`. This ensures that each pairwise force is computed only once, effectively halving the total number of iterations and improving performance. The number of loops decrease from N^2 to $0.5 \times N^2$, where the computation time decrease from 22,457 ms to 11,246 ms.

For loop unrolling, we also performed an experiment with an unroll factor of 2. However, the running time did not decrease. The main reason is that the dominant computation in the loop is the `sqrt()` operation. Any potential time savings from unrolling are smaller than the time spent on each `sqrt()`, so unrolling does not improve performance in this case. The pseudo code is shown in List 1 and the execution time is shown in Table 1.

```
for (int i = 0; i < N; i++)
{
    //Copy particle i's properties
    for (int j = i+1; j < N; j++)
    {
        // Compute forces symmetrically for both particles: F_ij = -F_ji
        // unroll 0
        f_x_ij[i] +=   F_x_0;
```

Table 1: Execution Time Comparison [ms]

Method	Time (ms)
Version 1	22,457
Symmetric Optimization	11,474
Loop Unroll	11,373
Constant Caching	11,246

```
        f_x_ij[i] +=   F_y_0;
        f_x_ij[j] += - F_x_0;
        f_y_ij[j] += - F_y_0;

        // unroll 1
        f_x_ij[i]   +=   F_x_1;
        f_x_ij[i]   +=   F_y_1;
        f_x_ij[j+1] += - F_X_1;
        f_y_ij[j+1] += - F_y_1;
    }
}
```

Listing 1: Optimized N-Body Force Calculation

2.3 Version 3 - Vectorization

For the earlier versions each time step introduced a call to memcpy to make the swap from the newly calculated state to the current state. For version 3 this feature was removed since whatever is the current state to be read from is stored inside the loop anyways. This change effectively reduces the memory complexity by $N \cdot 48 + 24$ bytes and doesnt waste time writing to memory two times.

```
        for (int step = 0; step < nsteps; step++)
        {
            update_particles(N, particles, particles_next, delta_t);
            memcpy(particles, particles_next, N * sizeof(Particle));
        }
```

Listing 2: Old memcpy

```
update_state(particle, N, ...,) {

    // Load current state
    state = particles.state

    // Calculate forces
    for i < N
        for i+1 < N
        new_state = update(state)
    particles.state = new_state
```

Listing 3: New logic

A third change was made which changed the structure of data from a an array of structures such as:

```
typedef struct {
    double pos_x;
    double pos_y;
    double mass;
    double vel_x;
    double vel_y;
    double brightness;
```

```
} Particle;
```

Listing 4: Old Array of Structures

To a structure of arrays:

```
typedef struct {
    double *x;
    double *y;
    double *mass;
    double *dx;
    double *dy;
    double *brightness;
} Gal_state;
```

Listing 5: New Structure of Arrays

These changes did not affect the performance noticeably. Which, in hindsight, is obvious since the expensive part of the computations can be attributed to the inner loop. A simple Valgrind check gave the following attempts to vectorize:

```
./galsim_v2.c:86:23:  missed:  couldn't vectorize loop
./galsim_v2.c:86:23:  missed:  not vectorized:  control flow in loop.
./galsim_v2.c:96:29:  missed:  couldn't vectorize loop
./galsim_v2.c:107:23:  missed:  not vectorized:  complicated access pattern.
```

Where line 86 corresponds to i loop, line 96 to j=i+1 loop and 107 to where the calculated force is written to memory.

An attempt to vectorize was made where the accumulated forces to particle i was stored in a local const variable inside:

```
    for (int i = 0; i < N; i++)
    {
        //Copy state of particle[i]

        // CHANGE: fxi, fyi accumulates the changes of
        // particle i
        double fxi = 0.0;
        double fyi = 0.0;
        for (int j = i+1; j < N; j++)
        {
            // Lots of expensive calculations

            // CHANGE: accumulate forces into fxi, fyi
            fxi += mass_j * div * r_x;
            fyi += mass_j * div * r_y;
            f_x_ij[j] += - mass_i * div * r_x;
            f_y_ij[j] += - mass_i * div * r_y;
        }

        // CHANGE - update the force state of particle i
        f_x_ij[i] += fxi;
        f_y_ij[i] += fyi;
```

Listing 6: Vectorization

This change resulted in the vectorization of the inner j=i+1 loop:

```
./galsim_v3.c:134:23:  missed:  couldn't vectorize loop
./galsim_v3.c:134:23:  missed:  not vectorized:  control flow in loop.
./galsim_v3.c:146:29:  optimized:  loop vectorized using 16 byte vectors
```

Line 134 is the outer i loop not being vectorized and 146 is the inner j=i+1 loop being vectorized. This change made a significantly speedup of the code.

3 The Parallel Solution

The overall flow of the pthread solution is shown in Figure 1. First, the workload is distributed among all threads. Each thread then computes the forces in parallel and updates the state of its assigned particles. Barrier synchronizations are used between phases to ensure correct execution order.

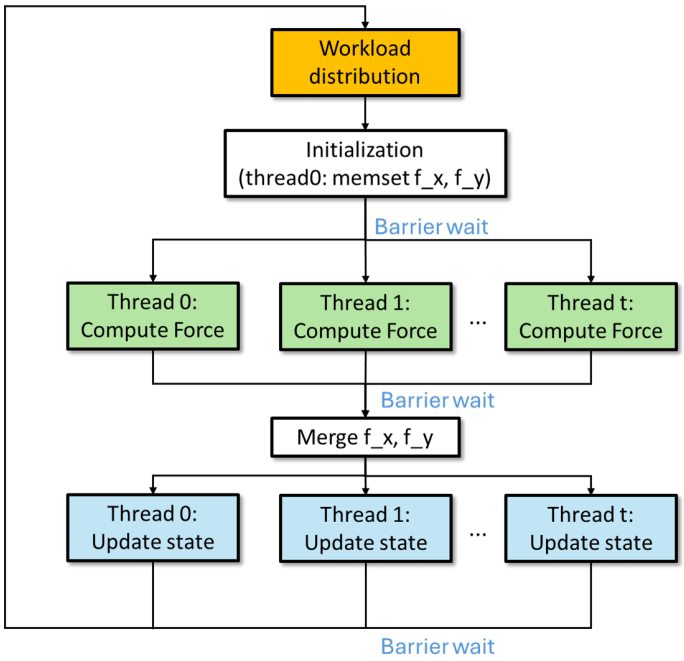

Figure 1: Pthread flow chart

3.1 Pthread: Workload imbalance

Two approaches was implemented to handle the work imbalance problem caused by the nature of the simulation. A contigious evenly balanced approach and a cyclic approach with chunk size of 1.

The first method of solving the problem of work imbalance was the contigious evenly balanced approach. We design an algorithm that distribute the work equally. The algorithm estimates the average workload per thread as

$$N(N-1)/(2T)$$

, where N is the number of particles and T is the number of threads. Each thread is assigned at least one rows and the accumulated number of pairwise interactions is approximately equal to this average workload. The Figure 2 provides an illustration of the thread workload distribution. The implementation is:

Implementation: Workload balancing

```
void compute_thread_range(int N, int n_thread, int i, int *start_idx, int *
    end_idx)
{
    float a = N * (N-1) / (2.0f * n_thread);
    int start = (*start_idx < N) ? *start_idx : N;
    int end = (*start_idx < N) ? *start_idx + 1 : N;
    int accumulate = (N - 1) - end;
```

```
    while (end < N) {
        if (accumulate >= a && i != n_thread - 1) break;
        end++;
        accumulate += N - 1 - end;
    }

    *start_idx = start;
    *end_idx = end;
}
```

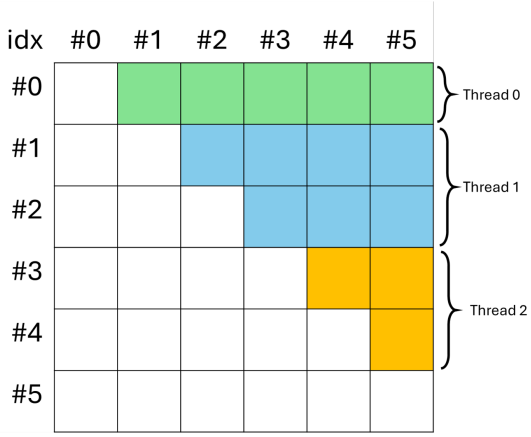

Figure 2: Illustration of workload balancing. The estimated average workload is 5. Each thread is assigned approximately the same amount of work, as shown by the different colors.

3.2 The second approach

The second approach was a simple cyclic implementation. Each thread is assigned a starting position and will step through the particles with a stride of length Nthreads. Effective read behaviour is like so:

$$[\text{Thrd1}][\text{Thrd2}]...[\text{ThrdN}][\text{Thrd1}][\text{Thrd2}]...[\text{ThrdN}][\text{Thrd1}]... \tag{1}$$

In practice this approach will not read from the particles array contiguously ultimately leading to more cachemisses. This problem can be alleviated by making the chunk sizes larger such that each chunk is contigious, say containing 8, 16 or 64 particles, which improves cache performance but at the same time makes the load balancing more uneven. In this report only the chunk size of 1 was implemented. This approach also have the benefit of having a smaller computational setup-cost and is simple to implement.

3.3 Pthread: Race condition

In the serial solution, the forces are accumulated in a global force array, and each particle can be processed independently. In the parallel solution, multiple threads updating the same array can lead to race conditions. To prevent this, we used thread-local arrays, mutexes, and barrier synchronizations to ensure correct accumulation and proper execution order.

Three barrier synchronizations are used in the pthread architecture. Thread 0 first initializes the global force arrays, and a barrier ensures all threads start after initialization. Then, all threads compute the forces in parallel, followed by a barrier to ensure completion. Finally, threads update the particle states in parallel, with a final barrier to synchronize before the next step.The pseudo code of parallel architecture is:

Implementation: Parallel Architecture

```
for each simulation step do

    Initialize global force arrays f_x and f_y
    Barrier()

    ComputeForcePerStep()
    Barrier()

    UpdateNextStep()
    Barrier()

end for
```

Since the overhead of mutex lock and unlock is non-negligible, a thread-local force array is created at each simulation step to reduce the number of mutex operations. Frequent locking can become a performance bottleneck. By using local arrays, the number of mutex operations is reduced to one per thread per step. At the end of the force computation, a single mutex is used to merge the local forces into the global arrays. The local arrays are allocated per step to ensure thread safety while keeping memory usage reasonable.

The pseudocode of the parallel force computation per step is as follows:

Implementation: Parallel Force Computation per Step

```
Procedure updateState()
    Malloc f_x_local and f_y_local
    for particle_idx: start_idx to end_idx:
        Compute pairwise interactions
        update f_x_local and f_y_local
    end for

    Mutex Lock
        Update global arrays:
            f_x += f_x_local
            f_y += f_y_local
    Mutex Unlock

    Free f_x_local, f_y_local
End Procedure
```

3.3.1 The second approach

The second approach handles the race conditions similarly as the first approach. One notable difference is that all memory allocations are initialized once in the main thread before execution. All private force arrays are stored in a double matrix of size Nthreads $\times$ Nparticles. This avoids the frequent calls to free() and calloc() that the first version utilize.

Another notable difference is that the updating of the shared force array is moved outside of the updating of the private force function. Hence, all threads update their private arrays, synchronize, then collectively update the shared array, synchronize and finally collectively update the next particle state. This has ultimately has the benefit of making it more straightforward remove the mutex dependence entirely.

Implementation: Parallel Force Computation per timestep

```
for each simulation step do

    update_private_forces()
    Barrier()
```

```
    consolidate_private_forces()
    Barrier()

    UpdateNextStep()
    Barrier()

end for
```

The consolidation phase is trivially parallel as each thread can be assigned a contigious chunk of columns to summarize the forces over. These reductions is then stored in the final force array. As the force for particle i has been computed the private force array is set to zero. Note that each thread will be responsible for a matrix of size Nthreads $\times$ chunksize. The matrix is accessed row-wise writing to the shared force array one particle at a time prompting better cache locality.

```
double *restrict fx = forces_x;

for (int j = 0; j < Nthreads; j++) {
    double *restrict rowx = Fx_mtrx[j];
    for (int i = start; i < stop; i++) {
        fx[i] += rowx[i];

        // can just reset to zero as we go,
        // avoiding memset/calloc going through the whole shebang again
        rowx[i] = 0.0;
    }
}
```

Finally the new particle state can be updated with the same scheduling as when consolidating forces. Similarly the indices of the shared force arrays are set to zero directly after they have been used.

3.4 OpenMP solution

The OpenMP solution uses an approach of parallelizing loops in the serial solution with the directive `#pragma omp parallel for`.

There are some loops in the serial solution that should not be parallelized, namely the `for` loop in the `read_state_config` function and in the `write_state`, since parallelization will lead to a mismatch between the original indices in the input file and the order of particles in the output file. Hence, the above two loops must remain serial.

The loop in the `update_state` function is broken into two parallelized loops in sequence. The first loop calculates the force between a pair of particles along the x-axis and the y-axis, and is parallelized with the OpenMP directive `#pragma omp parallel for num_threads(n_thread) reduction(+:f_x_ij[:N],f_y_ij[:N])`, with `reduction` configuring both force vectors, which are global sums, as shared variables that can only be updated by one thread at a time.

The second loop can only be started once all forces between a pair of particles along the x-axis and the y-axis have been calculated, and it calculates the overall force of a particle along the x-axis and the y-axis, so that its new velocity and position can be derived and updated.

Implementation: OpenMP Parallelization of State Update function

```
void update_state(int n_thread, const int N, Gal_state* restrict particles,
                  const double delta_t, const double G, const double epsilon,
                  double* restrict f_x_ij, double* restrict f_y_ij) {

    memset(f_x_ij, 0, N*sizeof(double));
    memset(f_y_ij, 0, N*sizeof(double));
```

```
    #pragma omp parallel for num_threads(n_thread) reduction(+:f_x_ij[:N],f_y_ij
   [:N])
    for (int i = 0; i < N; i++)
    {
        //Copy particle[i] to local variables avoiding reading memory each time
        const double pos_x_i = particles->x[i];
        const double pos_y_i = particles->y[i];
        const double mass_i = particles->mass[i];

        double fxi = 0.0;
        double fyi = 0.0;

        for (int j = i+1; j < N; j++)
        {
            const double mass_j = particles->mass[j];
            const double pos_x_j = particles->x[j];
            const double pos_y_j = particles->y[j];

            const double r_x = pos_x_i - pos_x_j;
            const double r_y = pos_y_i - pos_y_j;
            const double r_ij_mag = sqrt(r_x*r_x + r_y*r_y) + epsilon;
            const double div = 1/((r_ij_mag)*(r_ij_mag)*(r_ij_mag));

            const double fxj_decrement = mass_i * div * r_x;
            const double fyj_decrement = mass_i * div * r_y;

            // Update forces symmetrically for both particles: F_ij = -F_ji
            fxi += mass_j * div * r_x; //accumulate changes to i here instead of
   changing memory each iteration, this made it possible to vectorize
            fyi += mass_j * div * r_y;
            f_x_ij[j] -= fxj_decrement;
            f_y_ij[j] -= fyj_decrement;
        }

        f_x_ij[i] += fxi; // accumulated changes are written to i:th particle
        f_y_ij[i] += fyi;
    }

    #pragma omp parallel for num_threads(n_thread)
    for (int i = 0; i < N; i++)
    {
        const double pos_x_i = particles->x[i];
        const double pos_y_i = particles->y[i];
        const double mass_i = particles->mass[i];
        const double vel_x_i = particles->dx[i];
        const double vel_y_i = particles->dy[i];

        const double F_x = -G * mass_i * f_x_ij[i];
        const double F_y = -G * mass_i * f_y_ij[i];
        const double mass_i_div = 1.0f / mass_i;
        const double vel_x_next = vel_x_i + delta_t * F_x * mass_i_div;
        const double vel_y_next = vel_y_i + delta_t * F_y * mass_i_div; // last
   four definitons isnt used, save 4x mem?
        const double pos_x_next = pos_x_i + delta_t * vel_x_next;
        const double pos_y_next = pos_y_i + delta_t * vel_y_next;

        // update
        particles->x[i] = pos_x_next;
        particles->y[i] = pos_y_next;
        particles->dx[i] = vel_x_next;
        particles->dy[i] = vel_y_next;
    }
}
```

4 Serial Performance and Discussion

4.1 Method

This section presents how the timings were made. All code version have a similar pattern to them, with outline:

Example-code: How timing was done

```
int main(int argc, char *argv[]) {

    // Timing starts
    const double start = get_wall_seconds();

    //
    //      LOAD AND DEFINE RUNTIME VARIABLES
    //

    // Time-stepping
    for (int step = 0; step < nsteps; step++)
    {
        update_state(N, particles);
    }

    // free relevant memory
    free_state_memory(particles);
    particles = NULL;

    // Timing ends
    const double runtime = get_wall_seconds() - start;
```

The runtimes are calculated using get_wall_seconds() defined as:

Wall clock function [1]

```
double get_wall_seconds() {
    struct timeval tv;
    gettimeofday(&tv, NULL);
    return tv.tv_sec + (double)tv.tv_usec / 1000000;
}
```

Notice that the timing does not include writing the final state to a file. This is to reduce the impact from I/O-operations not directly relevant to the simulations. All three versions were timed five times for a set of different numbers of particles:

$$Ns = \{10, 20, 50, 70, 100, 200, 500, 700, 1000, 2000, 5000, 7000, 10000\}.$$

Additionally, each version was timed ten times for

$$Ns = 3000$$

using a total of 100 time steps in all cases. The mean and standard deviation of the measured runtimes were then calculated. Furthermore, all tests were made on a i7-11700k with the optimization flag -O3.

4.2 Performance results

The results of the three versions comparing runtimes for different amounts of particles are presented in figure 3. All versions follow a $O(N^2)$ curve albeit shifted differently. This is excpected since the

underlying algorithm used is $O(N^2)$. This plot clearly shows that version 3 of the code were always faster than both the 1st and 2nd version.

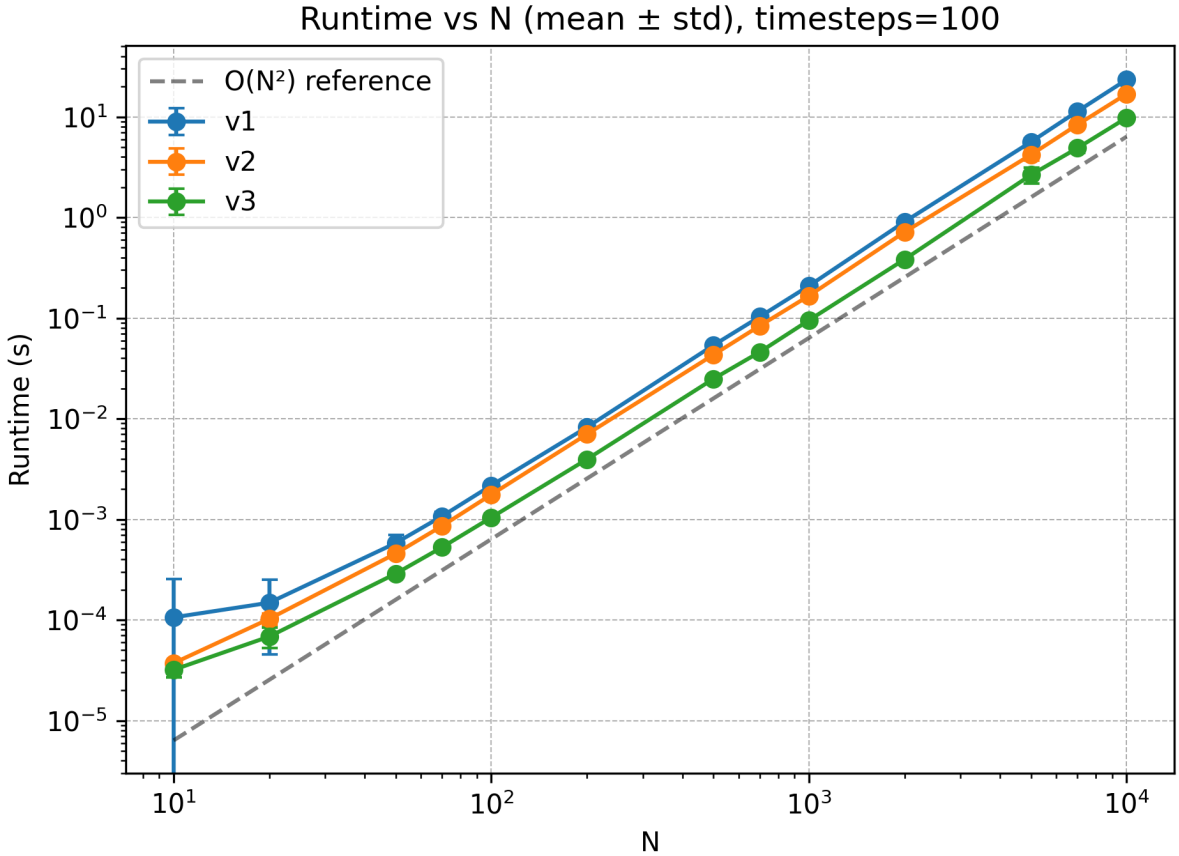

Figure 3: Loglog Runtimes vs Nparticles. Timings performed on an i7-11700k

The runtimes for the largest problem size are displayed in table 2. It shows that version v3 is 1.73 times faster than version v2 and 2.44 times faster than version v1. The v3 is the version that used vectorization for the inner j:th loop. The speedup from v1 to v2 of 1.41 times faster is also significant. The improvement here mainly being the symmetric update for the algorithm.

4.3 Performance Analysis

We used the tool perf to analyze where the most time is spent. In the annotated output shown in Figure 4, `sqrtpd` and `divpd` consumed 19.04% and 32.88% of CPU cycles, respectively, totaling over 50% of the execution time. This indicates that the algorithm has a clear bottleneck in calculating inter-particle distances and forces. Since the force computations for each particle pair are independent, parallelization can significantly improve performance by distributing subsets of particles across multiple threads, allowing better utilization of CPU cores.

Table 2: Runtime comparison for largest problem size ($N = 3000$, timesteps $= 100$). Timings performed on an i7-11700k

Version	**Runtime [s]**	**Relative speedup**[1]	**Total speedup**[2]
v1	2.109 ± 0.257	NaN	NaN
v2	1.496 ± 0.098	1.41	1.41
v3	0.862 ± 0.033	1.73	2.44

```
Percent      const double r_ij_mag = sqrt(r_x*r_x + r_y*r_y) + epsilon;
               movapd   %xmm6,%xmm4
  1,62         mulpd    %xmm6,%xmm4
               movapd   %xmm7,%xmm3
  0,02         mulpd    %xmm7,%xmm3
  0,07         addpd    %xmm4,%xmm3
 19,04         sqrtpd   %xmm3,%xmm3
  2,66         addpd    %xmm2,%xmm3
             const double div = 1/((r_ij_mag)*(r_ij_mag)*(r_ij_mag));
  0,90         movapd   %xmm3,%xmm4
  3,88         mulpd    %xmm3,%xmm4
  4,63         mulpd    %xmm3,%xmm4
               movapd   %xmm15,%xmm3
 32,88         divpd    %xmm4,%xmm3
             fxi += mass_j * div * r_x; //accumulate changes to i here in
  0,12         movupd   0x0(%r13,%rax,1),%xmm4
  6,85         mulpd    %xmm3,%xmm4
             f_x_ij[j] += - mass_i * div * r_x;
  1,65         mulpd    %xmm11,%xmm3
             fxi += mass_j * div * r_x; //accumulate changes to i here in
  0,02         movapd   %xmm4,%xmm8
  7,25         mulpd    %xmm6,%xmm8
             fyi += mass_j * div * r_y;
               mulpd    %xmm7,%xmm4
             f_x_ij[j] += - mass_i * div * r_x;
  1,31         mulpd    %xmm3,%xmm6
             f_y_ij[j] += - mass_i * div * r_y;
  1,51         mulpd    %xmm7,%xmm3
```

Figure 4: run time analysis

5 Parallel Performance and Comparison

This section presents a comprehensive performance evaluation of the parallel implementations and compares them to the optimized serial version. The objective is to analyze how effectively the N-body simulation utilizes multiple cores and to investigate the scalability of the algorithm.

The study includes measurements of runtime as a function of both problem size N and number of threads T. First, the scaling behavior with respect to N is examined in order to confirm the expected $\mathcal{O}(N^2)$ computational complexity. Thereafter, speedup plots are presented to evaluate how performance improves when increasing the number of threads. The achieved speedup is compared to the ideal linear speedup in order to assess parallel efficiency.

The speedup was measured for different numbers of threads $T = \{1, 2, 4, 8, 12, 16\}$ for N=5000 and 1000 with step size 100. These numbers were measured on an i7-11700k with 8 cores 16 threads in total. Compiler optimization flags used were -O3 -march=native -ffast-math -funroll-loops.

The achieved speedup is computed as

$$S(T) = \frac{T_{\text{serial}}}{T_{\text{parallel}}(T)},$$

where T_{serial} is the runtime of the optimized serial version and $T_{\text{parallel}}(T)$ is the runtime using T threads. Figure 5 shows the measured speedup together with the ideal linear speedup (dashed line), defined as $S_{\text{ideal}}(T) = T$.

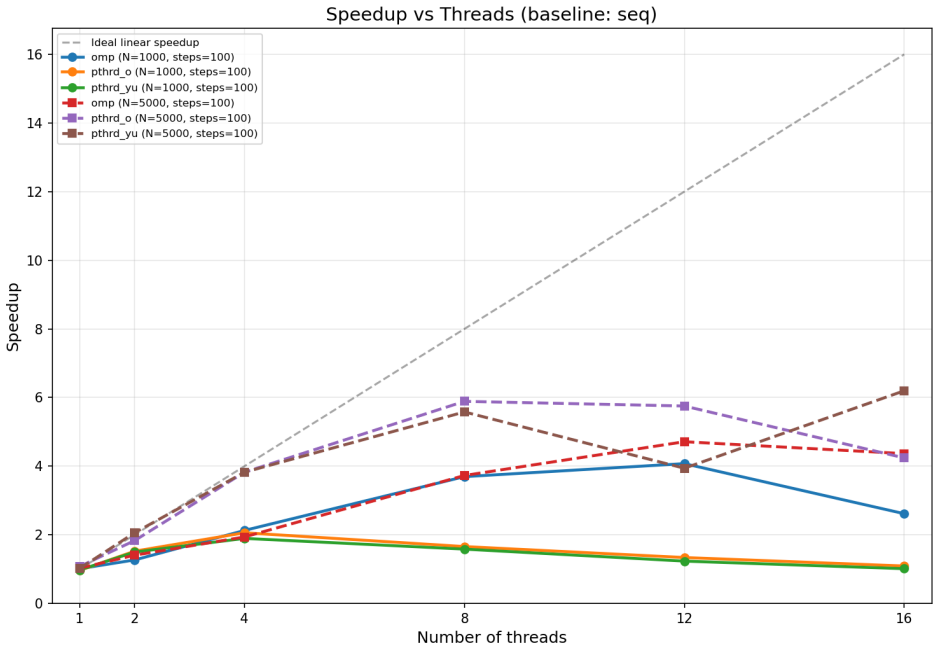

Figure 5: Speedup curves for all implementations. Speedup is calculated from the mean runtimes over a total of 20 runs.

Overall OpenMp performs better for all the smaller task size of N=1000 particles. Losing performance after 12 threads while the performance of both pthread solutions decrease after 4 threads. The pthread solutions for the smaller task and OpenMp solution for both tasks have a sublinear speedup. For the larger task both pthread solutions have a linear speedup up to 4 threads whereafter the performance stagnate and ultimately decrease. The pthread_o solution outperform the pthread_yu solution for 8 and 12 thread respectively, although the yu solution is slightly better at 2 threads and much better at 16 threads. The plots show that as the number of threads increase beyond the number of cores speedup tend to become worse.

5.1 Performance Comparison: Serial vs Parallel Versions

This subsection compares the optimized serial implementation with the best-performing parallel configurations. This test ran the different versions for various Nparticles ranging from 10 to 10k. Each run was performed 10 times for which mean runtime was calculated. A set of different thread settings were used Nthreads= 4,8,12,16.

The plot shows that for smaller problems OpenMP perform better. At some point all parallelized versions perform similarly on the grand scale, although perhaps in favour of the pthreaded programs. For really small problems 100 particles or less the sequential solution is the clear winner. The plots all show that eventually all solutions converge to the $\mathcal{O}(N^2)$ behaviour. At that point it is always better to parallelize than to run the code sequentially.

In figure 6

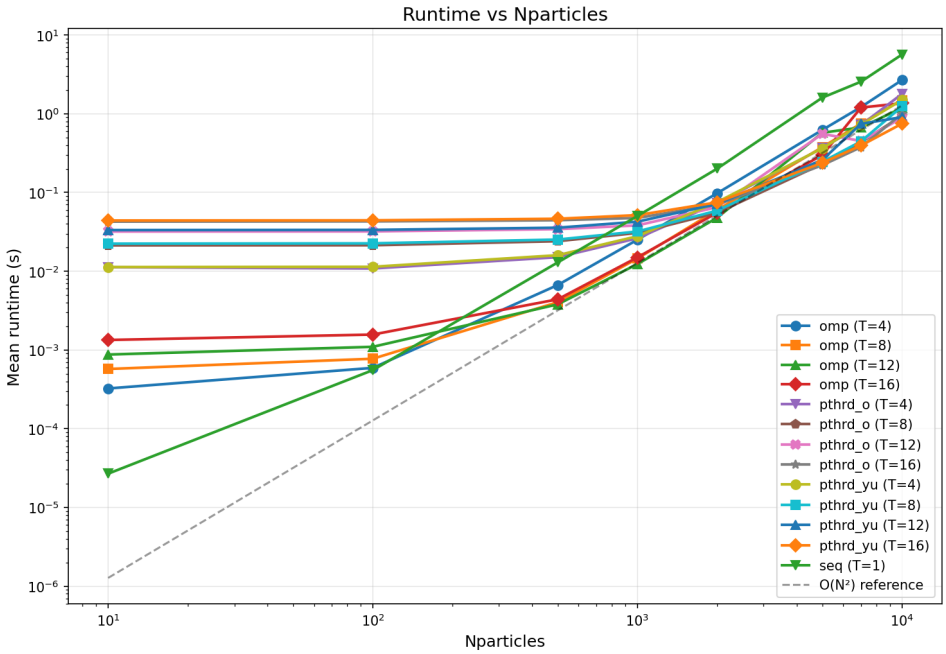

Figure 6: Loglog plot of mean runtimes across 10 runs for the different versions. A reference for the expected $\mathcal{O}(N^2)$ behaviour is included.

Another remark is that the OpenMp implementation can easily be made faster by creating a structure in which the threads are not respawned every timestep. This would most likely change the performance noticeably.

6 References

[1] HPP Course Material, "Code: get_wall_seconds()," 2026. Accessed from HPP course material.